Dokumentacja: Agent architecture & LLM
ForestFire

Autorzy:
Joanna Wojnowska, Antonina Wąsikowska

Kierunek studiów:
Informatyka i Systemy Inteligentne

Przedmiot:
Studio Projektowe - ForestFire

Prowadzący:
prof. dr. hab. Radosław Klimek

AGH
EAIiB
2026

Spis treści

1 Wprowadzenie **3**

2 Rozdzielenie Agent vs Logika Decyzyjna **3**

2.1 Filozofia architektury 3

3 Cykl życia agenta **3**

3.1 Czterofazowy model 3

3.2 Opis logiczny przepływu 4

4 Cykl życia agenta z integracją LLM **4**

4.1 Rozszerzony model DECIDE 4

4.2 Etapy przetwarzania rekomendacji LLM 5

5 Katalog decyzji per agent **6**

5.1 Wprowadzenie 6

5.2 FireBrigade (Brygada strażacka) 6

5.2.1 Decyzja #1: Wybór brygady do wysłania (LLM-eligible) 6

5.2.2 Decyzja #2: Priorytetyzacja pożarów (LLM-eligible) 7

5.3 ForesterPatrol (Patrol leśny) 8

5.3.1 Decyzja #3: Ocena zagrożenia (ThreatLevel) – kluczowy punkt LLM 8

5.4 Decyzje LLM-eligible 9

6 Architektura Strategy Pattern **10**

6.1 Koncepcja wzorca 10

6.2 Diagram klas – Brygady strażackie 11

6.3 Diagram klas – Patrole leśne 12

7 Mechanizm Fallback i Walidacji **12**

7.1 Filozofia systemu zabezpieczeń 12

7.2 Dwupoziomowa walidacja 13

7.3 Diagram procesu walidacji i fallback 14

7.4 Komponenty systemu walidacji 15

7.4.1 LLMResponseValidator 15

7.4.2 BrigadeStrategySelector i PatrolStrategySelector 15

7.5 Scenariusze aktywacji fallback 15

8 Runtime Switch API **15**

8.1 Koncepcja dynamicznego przełączania 15

8.2 Endpointy REST API 16

9 Podsumowanie **16**

9.1 Punkty integracji LLM 16

1 Wprowadzenie

Niniejszy dokument stanowi kompleksową dokumentację techniczną architektury agentów i modeli decyzyjnych w systemie ForestFire, ze szczególnym uwzględnieniem integracji z Large Language Models (LLM).

2 Rozdzielenie Agent vs Logika Decyzyjna

2.1 Filozofia architektury

System ForestFire opiera się na fundamentalnym rozdzieleniu dwóch warstw:

- **Warstwa wykonawcza (Agent)** – jednostka runtime działająca w symulacji
- **Warstwa decyzyjna (Strategy)** – wymienny algorytm generujący rekomendacje

Kluczowe rozróżnienie

- Agent **NIE podejmuje decyzji strategicznych**. Agent **WYKONUJE** otrzymane zalecenia.
- Moduł decyzyjny **NIE wykonuje akcji**. Moduł **REKOMENDUJE** akcje do wykonania.

3 Cykl życia agenta

3.1 Czterofazowy model

Każdy agent w systemie ForestFire działa w cyklu składającym się z czterech podstawowych faz. Model ten jest uniwersalny dla wszystkich typów agentów (brygad strażackich oraz patroli leśnych) i opisuje kompletny przepływ przetwarzania informacji w pojedynczym ticku symulacji.

Cykl życia agenta można zapisać w postaci sekwencji:

$$\textbf{SENSE} \rightarrow \textbf{DECIDE} \rightarrow \textbf{ACT} \rightarrow \textbf{REPORT}$$

Każda faza ma ściśle określoną odpowiedzialność i nie zachodzi między nimi mieszanie kompetencji.

3.2 Opis logiczny przepływu

W pojedynczym kroku symulacji agent wykonuje następujące działania:

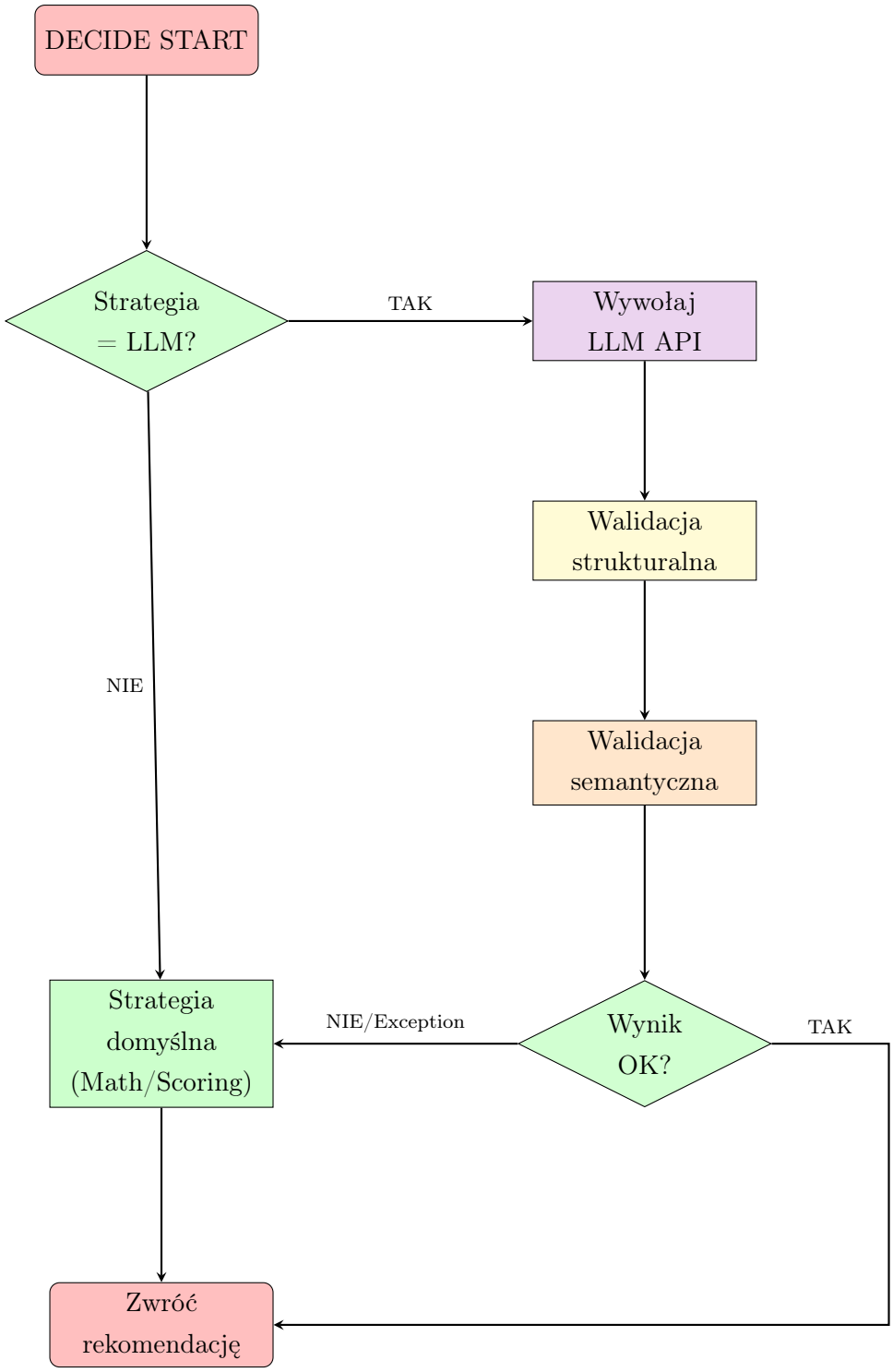
1. **SENSE (Percepcja)** – agent zbiera informacje o swoim stanie i otoczeniu.
2. **DECIDE (Decyzja)** – na podstawie aktualnego stanu wybierana jest akcja do wykonania.
3. **ACT (Akcja)** – agent wykonuje wybraną akcję w środowisku symulacyjnym.
4. **REPORT (Raport)** – agent raportuje swój stan do backendu.

Po zakończeniu fazy REPORT rozpoczyna się kolejny tick symulacji i cykl powtarza się od fazy SENSE.

4 Cykl życia agenta z integracją LLM

4.1 Rozszerzony model DECIDE

Gdy system korzysta ze strategii LLM, faza DECIDE staje się bardziej złożona i wymaga dodatkowych kroków walidacji oraz mechanizmu fallback.



Rysunek 1: Rozszerzony cykl DECIDE z integracją LLM i walidacją

4.2 Etapy przetwarzania rekomendacji LLM

Proces składa się z trzech kroków:

1. **Wywołanie LLM API** – backend przygotowuje snapshot stanu symulacji (agenty,

sektory, pożary), konstruuje prompt dostosowany do typu agenta i wysyła zapytanie z wymaganym formatem odpowiedzi JSON.

2. **Walidacja strukturalna** – weryfikacja poprawności formatu JSON, kompletności wymaganych pól oraz braku błędów API/timeout.

3. **Walidacja semantyczna** – komponent `LLMResponseValidator` sprawdza, czy wszystkie ID (`brigadeId`, `patrolId`, `sectorId`) istnieją w aktualnym stanie symulacji oraz czy zaproponowane akcje są wykonalne.

Walidacja semantyczna jest krytyczna: bez niej LLM mógłby zalecić wysłanie nieistniejącej brygady do nieistniejącego sektora. Każda rekomendacja musi zostać zweryfikowana przed przekazaniem do agenta.

5 Katalog decyzji per agent

5.1 Wprowadzenie

W tej sekcji przedstawiamy szczegółowy katalog wszystkich decyzji podejmowanych w systemie, z oznaczeniem, które z nich są kandydatami do realizacji przez LLM (LLM-eligible).

5.2 FireBrigade (Brygada strażacka)

Tabela 1: Katalog decyzji dla FireBrigade

#	**Decyzja**	**Typ**	**Gdzie**	**Obecna implementacja**	**LLM?**
1	Wybór brygady do wysłania	Taktyczna	Backend	MathDecisionStrategy	TAK
2	Priorytetyzacja pożarów	Taktyczna	Backend	Sortowanie po UrgencyIndex	TAK
3	Czy dotarłem do celu?	Lokalna	Agent	dist $<$ 0.002	NIE
4	Czy sektor płonie? $\rightarrow$ gaszenie	Lokalna	Agent	fire_level $>$ 0 && ACTIVE	NIE
5	Czy pożar ugaszony? $\rightarrow$ zakończ	Lokalna	Agent	is_task_finished()	NIE
6	Raportowanie stanu pożaru	Lokalna	Agent	Automatyczne do fire_event_queue	NIE
7	Zmiana stanu FSM	Lokalna	Agent	Deterministyczna maszyna stanów	NIE

5.2.1 Decyzja #1: Wybór brygady do wysłania (LLM-eligible)

Kontekst: System musi wybrać, która z dostępnych brygad zostanie wysłana do ugaszenia konkretnego pożaru.

Obecna implementacja: Algorytm `MathDecisionStrategy` oblicza `UrgencyIndex` dla każdego pożaru:

$$\text{UrgencyIndex} = w_1 \cdot \text{fireLevel} + w_2 \cdot \text{spreadRisk} - w_3 \cdot \text{distance} \tag{1}$$

gdzie w_1, w_2, w_3 to stałe wagi (hardcoded w kodzie).

Dlaczego LLM może być lepszy:

Aspekt	**Algorytm progowy**	**LLM**
Wagi	Stałe, niezależne od kontekstu	Dynamiczne, uwzględniają sytuację
Kontekst	Brak	Kierunek wiatru, zabudowania, pogoda, historia
Priorytetyzacja	Tylko matematyczna	„Infrastruktura krytyczna ma priorytet nad lasem”
Korelacje	Brak	„Te dwa pożary mogą się połączyć w większy”
Adaptacja	Stała formuła	Uczenie się z poprzednich sytuacji

5.2.2 Decyzja #2: Priorytetyzacja pożarów (LLM-eligible)

Kontekst: Gdy jest wiele aktywnych pożarów, system musi zdecydować, który z nich ma najwyższy priorytet.

Obecna implementacja: Sortowanie pożarów po obliczonym `UrgencyIndex`.

Dlaczego LLM może być lepszy: LLM może uwzględnić czynniki nieuchwytne w prostej formule matematycznej, takie jak:

- Bliskość zabudowań mieszkalnych
- Wartość chronionej infrastruktury
- Kierunek i prędkość wiatru
- Typ roślinności (łatwopalna vs trudnopalna)

5.3 ForesterPatrol (Patrol leśny)

Tabela 2: Katalog decyzji dla ForesterPatrol

#	**Decyzja**	**Typ**	**Gdzie**	**Obecna implementacja**	**LLM?**
1	Wybór sektora do patrolu	Taktyczna	Backend	ScoringBasedPatrolStrategy	TAK
2	Trasa patrolu	Taktyczna	Backend	Greedy po scoringu	TAK
3	**Ocena zagrożenia (ThreatLevel)**	Taktyczna	Backend	RiskCalculator + progi	TAK
4	Czy dotarłem do celu?	Lokalna	Agent	dist < 0.002	NIE
5	Wykrycie pożaru w sektorze	Lokalna	Agent	sector.fire_state == ACTIVE	NIE
6	Klasyfikacja intensywności	Lokalna	Agent	Progi: <25 MILD, <50 MOD, <75 FULL	NIE
7	Wysłanie alarmu pożarowego	Lokalna	Agent	Automatyczne do fire_event_queue	NIE
8	Czas trwania patrolu	Lokalna	Agent	patrol_duration = 7 ticks (konfig)	NIE
9	Zbieranie danych z sensorów	Lokalna	Agent	6 typów sensorów (automatyczne)	NIE
10	Zmiana stanu FSM	Lokalna	Agent	Deterministyczna maszyna stanów	NIE

5.3.1 Decyzja #3: Ocena zagrożenia (ThreatLevel) – kluczowy punkt LLM

Kontekst: Patrol zbiera dane z 6 typów sensorów i musi ocenić poziom zagrożenia pożarem w danym sektorze.

Obecna implementacja: Algorytm `RiskCalculator` z hardcoded wagami dla każdego sensora:

$$\text{risk} = w_T \cdot \text{norm}(T) + w_{RH} \cdot \text{norm}(RH) + w_W \cdot \text{norm}(W) + w_L \cdot \text{norm}(L) + w_{CO2} \cdot \text{norm}(CO2) + w_{PM} \cdot \text{norm}(PM \tag{2}$$

gdzie:

- $w_T = 0.15$ (waga dla temperatury)
- $w_{RH} = 0.15$ (waga dla wilgotności powietrza)
- $w_W = 0.10$ (waga dla wiatru)
- $w_L = 0.20$ (waga dla wilgotności ściółki)
- $w_{CO2} = 0.20$ (waga dla poziomu CO2 – dym)
- $w_{PM} = 0.20$ (waga dla PM2.5 – dym)

Dlaczego LLM może być lepszy:

Aspekt	**Algorytm progowy**	**LLM**
Wagi	Stałe przez cały rok	Dynamiczne (lato vs zima, dzień vs noc)
Kontekst	Brak	Pora roku, typ roślinności, historia sektora
Obserwacje tekstowe	Niemożliwe	„Widzę dym za wzgórzem”, „Sucha ściółka”
Korelacje przestrzenne	Brak	„Sektor 5 graniczy z płonącym sektorem 4”
Wykrywanie anomalii	Tylko progi	Rozpoznawanie nietypowych wzorców
Trend czasowy	Brak	„Temperatura rośnie od 3 godzin”

5.4 Decyzje LLM-eligible

Tabela 3: Lista wszystkich decyzji LLM-eligible

Agent	**Decyzja**	**Cel zastosowania LLM**
FireBrigade	Wybór brygady do wysłania	Optymalizacja przydziału zasobów z uwzględnieniem kontekstu
FireBrigade	Priorytetyzacja pożarów	Wielokryterialna analiza uwzględniająca wartość chronionego terenu
ForesterPatrol	Wybór sektora do patrolu	Adaptacyjny routing uwzględniający historię i ryzyko
ForesterPatrol	Trasa patrolu	Optymalizacja pokrycia terenu z uwzględnieniem zagrożeń
ForesterPatrol	Ocena zagrożenia (ThreatLevel)	Interpretacja danych sensorowych w kontekście czasowym i przestrzennym

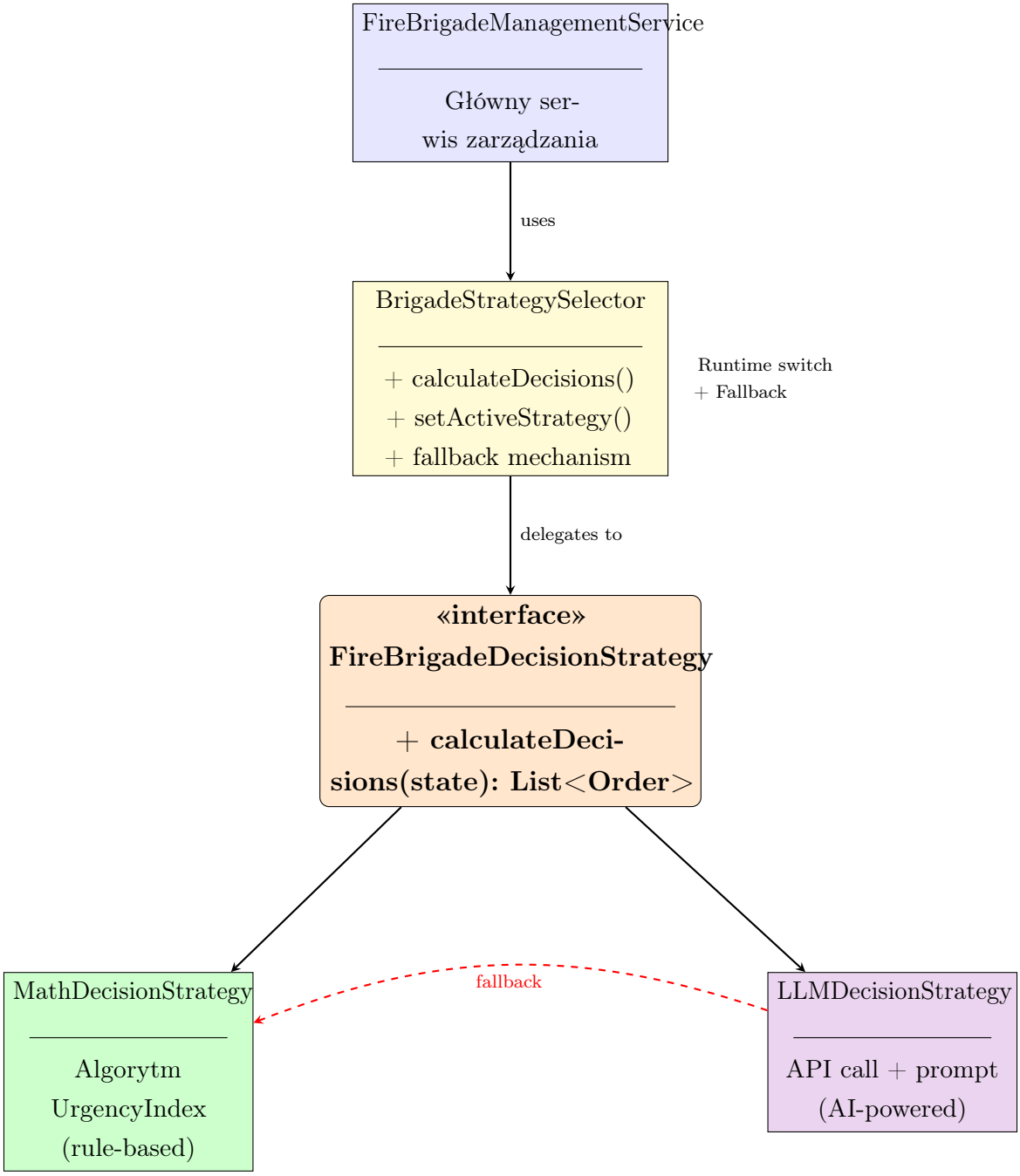
6 Architektura Strategy Pattern

6.1 Koncepcja wzorca

System ForestFire implementuje klasyczny wzorzec projektowy **Strategy Pattern**, który pozwala na:

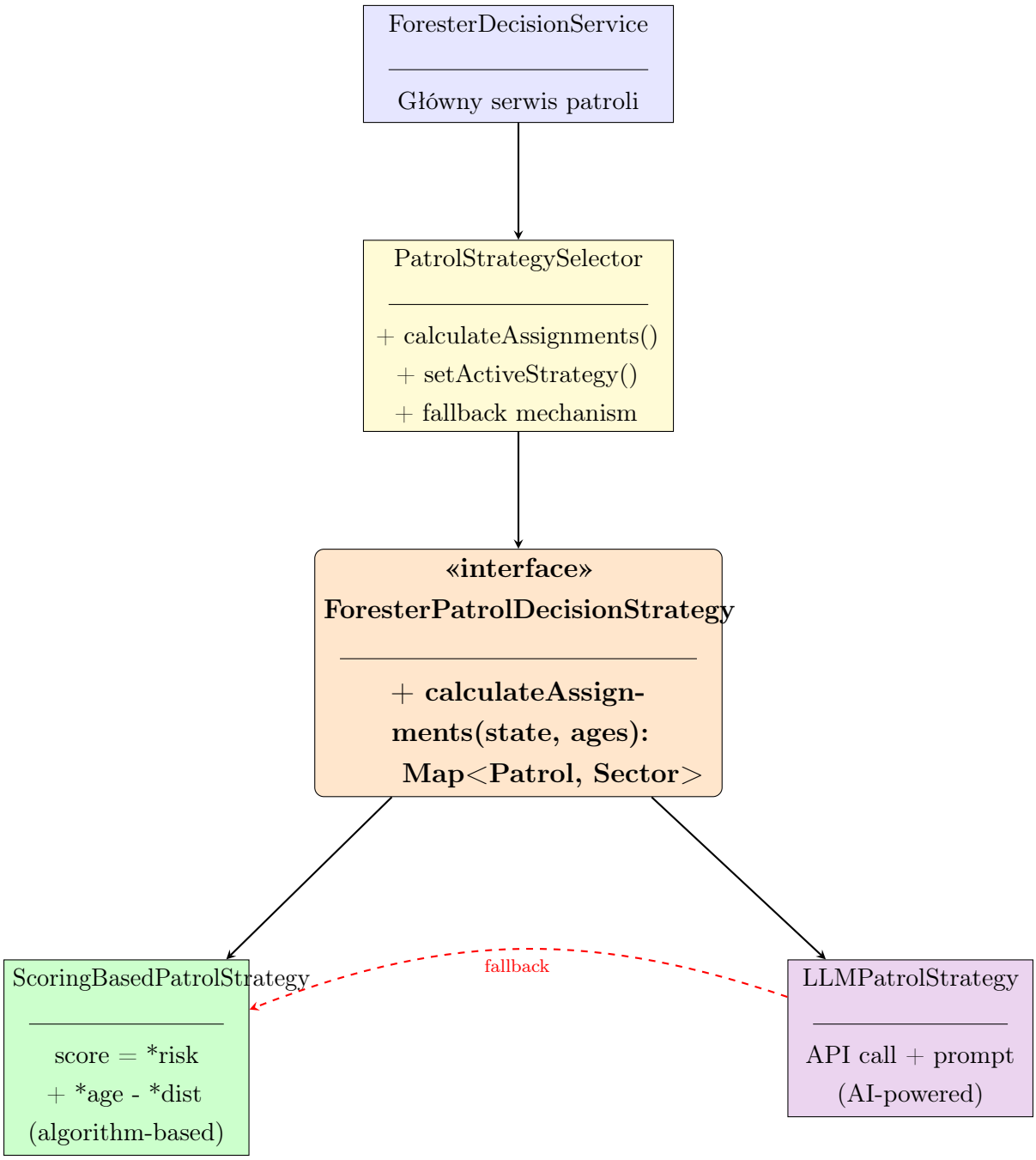
- Wymienność algorytmów decyzyjnych bez modyfikacji kodu agentów
- Łatwe dodawanie nowych strategii (np. algorytmy MCTS, genetyczne)
- Runtime switch – zmianę strategii w trakcie działania systemu
- Mechanizm fallback – automatyczne przejście na strategię zapasową

6.2 Diagram klas – Brygady strażackie



Rysunek 2: Strategy Pattern dla brygad strażackich

6.3 Diagram klas – Patrole leśne



Rysunek 3: Strategy Pattern dla patroli leśnych

7 Mechanizm Fallback i Walidacji

7.1 Filozofia systemu zabezpieczeń

System ForestFire implementuje podejście **"never trust, always verify"** w kontekście rekomendacji LLM. Każda odpowiedź z modelu językowego przechodzi przez dwupoziomowy

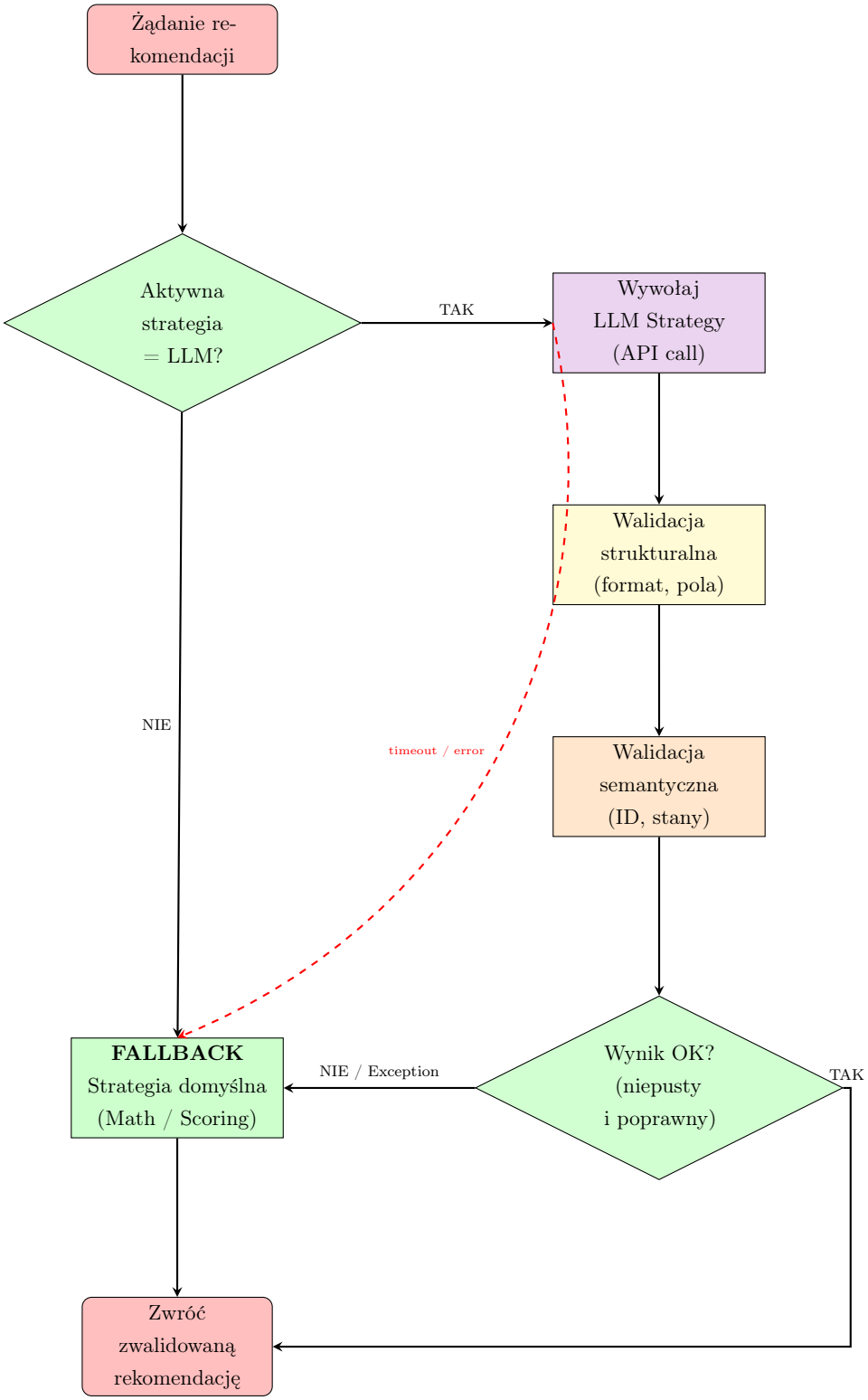
proces weryfikacji.

7.2 Dwupoziomowa walidacja

Każda rekomendacja LLM przechodzi przez:

1. **Walidacja strukturalna** – poprawność formatu JSON, kompletność pól, brak błędów API
2. **Walidacja semantyczna** – istnienie ID obiektów, zgodność z aktualnym stanem symulacji, brak konfliktów

7.3 Diagram procesu walidacji i fallback



Rysunek 4: Mechanizm fallback z dwupoziomową walidacją

7.4 Komponenty systemu walidacji

7.4.1 LLMResponseValidator

Centralny komponent weryfikujący sensowność rekomendacji LLM: sprawdza istnienie ID agentów i sektorów, dostępność jednostek oraz filtruje niepoprawne zalecenia. Szczegółowe logi pozwalają na śledzenie odrzuconych rekomendacji.

7.4.2 BrigadeStrategySelector i PatrolStrategySelector

Selektory implementują mechanizm wyboru strategii z automatycznym fallback:

- Jeśli LLM zwróci poprawne rekomendacje → użyj ich
- Jeśli LLM zawiedzie → automatyczne przełączenie na Math/Scoring
- Pełna implementacja w sekcji 6

7.5 Scenariusze aktywacji fallback

Tabela 4: Sytuacje aktywacji mechanizmu fallback

Sytuacja	**Fallback?**
LLM zwrócił pustą odpowiedź (`null` lub `[]`)	TAK
LLM timeout lub błąd API (500, connection refused)	TAK
LLM nieprawidłowy format JSON	TAK
LLM zwrócił nieistniejące ID (brigade, patrol, sector)	TAK
Wszystkie rekomendacje LLM odrzucone podczas walidacji	TAK

8 Runtime Switch API

8.1 Koncepcja dynamicznego przełączania

System ForestFire oferuje REST API pozwalające na **dynamiczną zmianę strategii decyzyjnej bez restartu** aplikacji. Jest to kluczowe dla:

- Eksperymentów porównawczych (LLM vs algorytmy)
- A/B testingu w czasie rzeczywistym

- Reagowania na awarie LLM
- Optymalizacji kosztów (włączenie LLM tylko w krytycznych momentach)

8.2 Endpointy REST API

Tabela 5: Endpointy API przełączania strategii

Method	**Endpoint**	**Opis**
GET	/api/strategy	Zwraca status wszystkich strategii (brygady + patrole)
GET	/api/strategy/brigades	Zwraca status strategii brygad strażackich
PUT	/api/strategy/brigades?type=MATH|LLM	Przełącza strategię brygad na MATH lub LLM
GET	/api/strategy/patrols	Zwraca status strategii patroli leśnych
PUT	/api/strategy/patrols?type=SCORING|LLM	Przełącza strategię patroli na SCORING lub LLM

9 Podsumowanie

9.1 Punkty integracji LLM

System identyfikuje **5 kluczowych punktów integracji LLM**:

1. **Wybór brygady do wysłania** (FireBrigade)
   - Optymalizacja przydziału zasobów
   - Uwzględnienie kontekstu sytuacyjnego
2. **Priorytetyzacja pożarów** (FireBrigade)
   - Wielokryterialna analiza
   - Wartość chronionego terenu
3. **Wybór sektora do patrolu** (ForesterPatrol)
   - Adaptacyjny routing
   - Historia zagrożeń

4. **Trasa patrolu** (ForesterPatrol)
   - Optymalizacja pokrycia terenu
   - Minimalizacja czasu reakcji
5. **Ocena zagrożenia (ThreatLevel)** (ForesterPatrol)
   - Interpretacja danych sensorowych
   - Kontekst czasowy i przestrzenny
   - **Największy potencjał LLM**